

```
import math
```

FUNCTION TO PRINT A MATRIX

```
def print_matrix(matrix, title="Matrix"):
```

```
    print("\n" + title + ":")

    for row in matrix:
        for value in row:
            print(f"{value:8.2f}", end=" ")
        print()
```

FUNCTION TO VALIDATE INPUT

```
def validate_input(A, b):
```

```
    # Check if matrix is empty
    if len(A) == 0:
        raise ValueError("Coefficient matrix cannot be empty.")

    n = len(A)

    # Check if matrix is square
    for row in A:
        if len(row) != n:
            raise ValueError(
                "Coefficient matrix must be square."
            )

    # Check RHS size
    if len(b) != n:
        raise ValueError(
```

```
        "RHS vector size must match matrix size."  
    )
```

FUNCTION TO CREATE AUGMENTED MATRIX

```
def create_augmented_matrix(A, b):
```

```
    augmented = []  
  
    for i in range(len(A)):  
  
        row = []  
  
        # Add elements of A  
        for j in range(len(A[i])):  
            row.append(float(A[i][j]))  
  
        # Add RHS value  
        row.append(float(b[i]))  
  
        augmented.append(row)  
  
    return augmented
```

GAUSSIAN ELIMINATION

WITH PARTIAL PIVOTING

```
def gaussian_elimination(A, b):
```

```
    tolerance = 1e-10
```

```

# Validate input
validate_input(A, b)

# Create augmented matrix
augmented = create_augmented_matrix(A, b)

n = len(A)

# Go through each column
for i in range(n):

    # -----
    # FIND THE BEST PIVOT
    # -----

    pivot_row = i

    for j in range(i + 1, n):

        if abs(augmented[j][i]) > abs(augmented[pivot_row][i]):
            pivot_row = j

    # Check if pivot is zero
    if abs(augmented[pivot_row][i]) < tolerance:
        continue

    # -----
    # SWAP ROWS
    # -----

    if pivot_row != i:

        temp = augmented[i]
        augmented[i] = augmented[pivot_row]
        augmented[pivot_row] = temp

    # -----
    # ELIMINATION
    # -----

    for j in range(i + 1, n):

        # If value is already zero, skip
        if abs(augmented[j][i]) < tolerance:
            augmented[j][i] = 0
            continue

        # Calculate elimination factor
        factor = augmented[j][i] / augmented[i][i]

        # Subtract pivot row
        for k in range(i, n + 1):

            augmented[j][k] = (

```

```

        augmented[j][k]
        - factor * augmented[i][k]
    )

    # Remove very small floating-point errors
    if abs(augmented[j][i]) < tolerance:
        augmented[j][i] = 0

# -----
# CHECK FOR SINGULAR / INCONSISTENT SYSTEM
# -----

for i in range(n):

    all_zero = True

    for j in range(n):

        if abs(augmented[i][j]) >= tolerance:
            all_zero = False
            break

    # Example:
    # 0 0 0 | 5
    # means no solution

    if all_zero and abs(augmented[i][n]) >= tolerance:
        raise ValueError(
            "System is inconsistent and has no solution."
        )

    # Example:
    # 0 0 0 | 0
    # means infinitely many solutions

    if all_zero and abs(augmented[i][n]) < tolerance:
        raise ValueError(
            "System does not have a unique solution."
        )

return augmented

```

BACK SUBSTITUTION

```
def back_substitution(upper_matrix):
```

```
tolerance = 1e-10

n = len(upper_matrix)

# Create solution array
x = []

for i in range(n):
    x.append(0.0)

# Start from last equation
for i in range(n - 1, -1, -1):

    # Get pivot
    pivot = upper_matrix[i][i]

    # Check for zero pivot
    if abs(pivot) < tolerance:
        raise ValueError(
            "Zero pivot encountered during back substitution."
        )

    # Start with RHS value
    sum_value = upper_matrix[i][n]

    # Subtract already known values
    for j in range(i + 1, n):

        sum_value = (
            sum_value
            - upper_matrix[i][j] * x[j]
        )

    # Calculate unknown
    x[i] = sum_value / pivot

return x
```

VERIFY SOLUTION

```
def verify_solution(A, b, x):
```

```
tolerance = 1e-8

calculated = []

# Calculate A × x
for i in range(len(A)):

    value = 0

    for j in range(len(x)):

        value = value + A[i][j] * x[j]

    calculated.append(value)

# Compare calculated values with b
verified = True

for i in range(len(b)):

    if abs(calculated[i] - b[i]) > tolerance:
        verified = False
        break

return verified, calculated
```

COMPLETE SOLVER

```
def solve_system(A, b):
```

```
    print_matrix(
        A,
        "Input Coefficient Matrix"
    )

    print("\nRHS Vector:")
    print(b)

    # Create and display augmented matrix
    augmented = create_augmented_matrix(A, b)

    print_matrix(
```

```
        augmented,
        "Augmented Matrix [A | b]"
    )

# Gaussian elimination
upper_matrix = gaussian_elimination(A, b)

print_matrix(
    upper_matrix,
    "Matrix After Forward Elimination"
)

# Back substitution
solution = back_substitution(upper_matrix)

print("\nSolution:")

for i in range(len(solution)):

    print(
        "x" + str(i + 1) + " = "
        + f"{solution[i]:.2f}"
    )

# Verification
verified, calculated = verify_solution(
    A,
    b,
    solution
)

print("\nVerification:")

print("Calculated A x x:")

for value in calculated:
    print(f"{value:.2f}", end=" ")

print("\nOriginal RHS b:")

for value in b:
    print(f"{value:.2f}", end=" ")

print()

if verified:
    print("\nResult: Solution verified successfully.")
else:
    print("\nResult: Verification failed.")

return solution
```

=====

=====

TEST SYSTEM 1

=====

=====

```
A1 = [ [2, 1, -1], [-3, -1, 2], [-2, 1, 2] ]
```

```
b1 = [8, -11, -3]
```

```
print("\n" + "=" * 60) print("SYSTEM 1") print("=" * 60)
```

try:

```
solution1 = solve_system(A1, b1)
```

except ValueError as error:

```
print("Error:", error)
```

=====

=====

TEST SYSTEM 2

=====

=====

```
A2 = [ [1, 2, 3], [2, -1, 1], [3, 1, -2] ]
```

```
b2 = [14, 3, -1]
```

```
print("\n" + "=" * 60) print("SYSTEM 2") print("=" * 60)
```

try:


```
solution2 = solve_system(A2, b2)
```

except ValueError as error:

```
print("Error:", error)
```

```
=====
```

TEST SYSTEM 3

```
=====
```

```
A3 = [ [2, 1], [1, 3] ]
```

```
b3 = [5, 6]
```

```
print("\n" + "=" * 60) print("SYSTEM 3") print("=" * 60)
```

try:

```
solution3 = solve_system(A3, b3)
```

except ValueError as error:

```
print("Error:", error)
```